

Generation of Elementary Discrete Time Sequences

Aim:

To generate discrete time sequences using MATLAB.

Requirements:

Matlab software

Pc

Discrete Time:

The signals that are defined for discrete instant of time are known as discrete time signals. They are continuous in amplitude and discrete in time.

Sinusoidal signal:

It is mathematically expressed as

$$x(t) = \sin(2\pi f t)$$

where f is the frequency, t is the time.

Exponential Signal:

The continuous time exponential signal can be expressed as follows

$$x(t) = ce^{at}$$

where, c and a are in general numbers

Delta Function:

It is mathematically defined as the signal which has amplitude of one only at $t=0$

$$\delta(n) = 1 \quad \text{for } t = 0$$

$$\delta(n) = 0 \quad \text{for } t \neq 0$$

Procedure:

Open the MATLAB software

Create the new empty file

Type the required program

Save it in the current directory

Save the output

End the program

Program:

Sine wave:

$$x = -30 : 0.5 : 30;$$

$$y = 5 * \sin(2 * 3.14 * x * 100);$$

subplot(2, 1, 1); stem(x, y)

xlabel('a(n) →'); ylabel('amplitude →')

title('sine signal digital');

subplot(2, 1, 2); plot(x, y)

xlabel('b(t)'); ylabel('amplitude →')

title('sine signal analog');

Exponential signal:

$$x = -3 : 0.1 : 3;$$

$$y = \exp(x/2);$$

subplot(2, 1, 1); stem(x, y)

xlabel('a(n) →'); ylabel('amplitude →')

title('exponential signal digital');

subplot(2, 1, 2); plot(x, y)

xlabel('a(t)'); ylabel('amplitude →')

title('exponential signal analog');

Delta signal:

```
x = -4:0.01:4;
```

```
y = [x==0];
```

```
plot(x,y); xlabel('t) →'); ylabel('amplitude  
→')
```

```
title('delta signal')
```

Result:

Thus the MATLAB code for generating sine, exponential and delta waveform (analog and digital) was performed and simulated and the output is obtained.

Date:

Aim:

To write and simulate the MATLAB code for generating linear convolution and circular convolution

Apparatus required:

PC (personal computer)

MATLAB software

Theory:Convolution:

Convolution is a mathematical way of combining two signals to form a third signal.

Linear Convolution:

Linear convolution is a mathematical operation used to express the relation between input and output of an LTI system

$$y(n) = x(n) * h(n)$$

where $y(n)$ = output of LTI and
 $x(n)$ = input of LTI

Circular Convolution:

The circular convolution is also known as cyclic convolution of two aperiodic function occurs when one of them is converted in the normal way with a periodic summation of the other function. The DFT of the periodic convolution of DFT of the individual sequences.

Procedure:

Open the MATLAB software

Create the new empty file

Type the required program

Save it in the current directory

Save the output

End the program

Program:

Linear Convolution:

```
x = input('enter the sequence 1:');  
h = input('enter the sequence 2:');  
y = conv(x, h);  
subplot(3, 1, 1);  
stem(x);  
ylabel('amplitude  $\rightarrow$ ');  
xlabel('N  $\rightarrow$ ');  
title('input sequence x');  
subplot(3, 1, 2);  
stem(h);  
ylabel('amplitude  $\rightarrow$ ');  
xlabel('N  $\rightarrow$ ');  
title('input sequence h');  
subplot(3, 1, 3);  
stem(y);  
ylabel('amplitude  $\rightarrow$ ');  
xlabel('N  $\rightarrow$ ');  
title('linear convolution');
```

Circular convolution:

```
g = input('enter the sequence 1:');  
h = input('enter the sequence 2:');  
n1 = length(g);  
n2 = length(h);  
n = max(n1, n2);  
n3 = n1 - n2;  
if (n3 > 0)  
    h = [h, zeros(1, n3)];  
else  
    g = [g, zeros(1, -n3)];  
end  
for n = 1:n;  
    y(n) = 0;  
    for i = 1:n;  
        j = n - i + 1;  
        if (j <= 0)  
            j = n + j;  
        end
```

```

y(n) = [y(n) + (g(i) * h(j))];
end
end
disp('The resultant is'); y
stem(y);
xlabel('n ->');
ylabel('amplitude ->');

```

Result:

Thus the MATLAB code for linear convolution and circular convolution was performed and simulated and the output is obtained.

Date:

Aim:

To write and simulate the MATLAB code for generating auto correlation and cross correlation

Apparatus required:

PC (personal computer)

MATLAB software

Theory:Correlation:

Correlation is a statistical measure that express the extent to which two variable are linearly related (meaning they change together at a constant rate). It is a common tool for describing sample relationship without making a statement about cause and effect.

Difference Between auto correlation and cross correlation:

Correlation is a statistical measure that expresses that extent to which two variables are linearly related (meaning they change together at a constant scale) it's a common tool for describing simple relationship without making a statement about Cause and effect.

Procedure:

Open the MATLAB software
Type the required program
Save it in the current directory
Save the output
End the program.

Program:Auto correlation:

```
clc;  
close all;
```

```

clear all;
x = input('enter the sequence 1:');
y = x corr(x, x);
Figure;
subplot(2, 1, 1);
stem(x);
ylabel('Amplitude  $\rightarrow$ ');
xlabel('n  $\rightarrow$ ');
title('input sequence');
subplot(2, 1, 2);
stem(fliplr(y));
xlabel('amplitude');
xlabel('n  $\rightarrow$ ');
title('output sequence');
disp('the resultant is');
fliplr(y);

```

Cross Correlation:

```

clear;
clear all;
close all;
x = input('Enter the sequence 1:');
h = input('Enter the sequence 2:');
y = x corr(x, h);
Figure;
subplot(3, 1, 1);
stem(x);
xlabel('n  $\rightarrow$ ');
ylabel('amplitude  $\rightarrow$ ');
title('Input sequence 1');
subplot(3, 1, 2);
stem(fliplr(y));
stem(h);
xlabel('n  $\rightarrow$ ');
ylabel('Amplitude  $\rightarrow$ ');
title('input sequence 2');
disp('The resultant is');
fliplr(y);

```


Result:

Thus the MATLAB code for generating correlations signal (analog and digital) was performed and simulated and the output is obtained.

Ex: No:

Frequency analysis using DFT

Date:

Aim:

To write and simulate the MATLAB code for spectrum analysis using DFT

Apparatus required:

PC (personal computer)

MATLAB software

Theory:

DFT:

The discrete fourier transformer (DFT) implemented by one of the computationally efficient fast fourier transform (FFT) algorithms, has become the core of many digital signal processing systems. These system can perform general time domain signal processing and classical frequency domain processing.

The goal of spectrum analysis is often to determine the frequency content of an analog (continuous time) signal, very often as in most modern spectrum analyzers, this is actually accomplished by sampling the analog signal.

Windowing (truncating) the data and computing and plotting the magnitude of its DFT.

Procedure:

Open the MATLAB software
Create the new empty file
Type the required program
Save it in the current directory
Save the output
End the program

Program:

```
x = [1 2 4 5 6];  
subplot (3,1,1);  
stem(x);  
xlabel ('Time →') ylabel ('amplitude →')  
title ('input sequence');  
y = fft(x);  
subplot (3,1,2);  
disp(y);  
xlabel ('Real part →');  
ylabel ('img part →');  
stem (real(y), imag(y));  
xlabel ('real part →'); ylabel ('img part →');  
stem (real (y); imag (y));  
title ('dft of input sequence');  
z = ifft (y);  
subplot (3,1,3);  
disp(z);  
xlabel ('real part →');  
ylabel ('img part →');  
stem (z);  
title ('idft of fft sequence')
```

Result:

Thus the MATLAB code for spectrum analysis using DFT was performed and simulated and the output is obtained.

Ex. No: Design of FIR filters (LPE / HPE / BPE / BSE)
and Demonstrates the filtering operation.

Date:

Aim: To write and simulate the MATLAB code for FIR filter design.

Apparatus required:

FIR

Finite impulse response converts infinite number of sequences into a finite number of sequence. FIR filter design uses the following windows.

Rectangular window:

The width of main pole in window spectrum is $4/N$

In FIR filter designed using rectangular window the minimum stop band attenuation is 22db

$$w(n) = 1 \text{ for } -(N-1/2) \text{ to } (N-1/2)$$

$$w(n) = 0 \text{ for other } n.$$

Hanning window:

The width of the main pole in window spectrum is $8/N$. In FIR filter designed using rectangular window the minimum stop band attenuation is 44db

$$w(n) = 0.5 - 0.5 \cos\left(\frac{2\pi n}{N} - 1\right) \text{ for } -(N-1/2) \text{ to } (N-1/2)$$

$$w(n) = 0 \text{ for other } n$$

Hamming window:

The width of the main pole in window spectrum is $8/N$. In FIR filter designed using rectangular window the minimum stop band attenuation is 51db

$$w(n) = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N} - 1\right) \text{ for } -(N-1/2) \text{ to } (N-1/2)$$

$$w(n) = 0 \text{ for other } n$$

High pass filter:

The high pass filter allows the frequencies below the cutoff frequency and rejects the frequency lower than the first off frequency.

Band pass filter:

The band pass filter passes the frequencies within the range ω_1 and ω_2 , and rejects the frequencies lower outside the range.

Band stop filter or Band rejection filter:

The band stop filter or the band rejection filter is a filter that passes most frequencies unaltered. But attenuates these in a specific range to very low levels it is the opposite of a band pass filter. A notch filter is a band filter with narrow stop band.

Low pass filter:

The low pass filter allows the frequencies below the cutoff frequency and rejects the frequencies greater than the cutoff frequency.

Procedure:

Open the MATLAB software
Create the new empty file
Type the required program
Save it in the current directory
Save the output
End the program.

Program:

low pass filter:

```
clc;  
Wc = 0.5 * Pi;  
N = 25;  
alpha = (N-1)/2;  
eps = 0.001;
```



```

n = 0:1:N-1;
hd = sin (wc * (n-alpha + eps)) ./ (pi * (n-alpha + eps));
Wt = boxcar(N);
hn = hd * Wt';
w = 0:0.01:pi;
h = freqz(hn, 1, w);
plot (w/pi, abs(h));
xlabel ('normalised frequency');
ylabel ('magnitude');
hold on;
Wh = hamming(N);
hn = hd * Wh';
w = 0:0.01:pi;
h = freqz(hn, 1, w);
plot (w/pi, abs(h) * -1);
grid;
hold off;

```

High pass filter:

```

clc;
wc = 0.5 * pi;
N = 25;
alpha = (N-1)/2;
eps = 0.001;
n = 0:1:N-1;
hd = (sin (pi * (n-alpha + eps)) - sin (wc * (n-alpha + eps))) ./ (pi * (n-alpha + eps));
Wt = boxcar(N);
hn = hd * Wt';
w = 0:0.01:pi;
h = freqz(hn, 1, w);
plot (w/pi, abs(h));
xlabel ('normalised frequency');
ylabel ('magnitude');
hold on;

```



```

wh = hanning(w);
hn = hd * wh;
w = 0:0.01:Pi;
h = freqz(hn, 1, w);
plot(w/Pi, abs(h), '-');
grid;
hold off;

```

Band pass filter:

```

clc;
wc1 = 0.25 * Pi;
wc2 = 0.75 * Pi;
N = 25; alpha = (N-1)/2;
eps = 0.001;
n = 0:1:N-1;
hd = (sin(wc2 * (n-alpha + eps)) - sin(wc1 * (n-alpha + eps))) /
      (Pi * (n-alpha + eps));

```

```

wtr = boxcar(N);
hn = hd * wtr;
w = 0:0.01:Pi;
h = freqz(hn, 1, w);
plot(w/Pi, abs(h));
xlabel('normalised frequency');
ylabel('magnitude e');

```

```

hold on;
wh = hamming(N);
hn = hd * wh;
w = 0:0.01:Pi;
h = freqz(hn, 1, w);
plot(w/Pi, abs(h), '-');
wh = hamming(N);
hn = hd * wh;
w = 0:0.01:Pi;
h = freqz(hn, 1, w);
plot(w/Pi, abs(h), '-');
grid;

```

hold off;
Band stop filter:

```
clc;  
wc1 = 0.25 * pi;  
wc2 = 0.75 * pi;  
N=25;  
alpha = (N-1)/2;  
eps = 0.001;  
n = 0:1:N-1;  
hd = sin(wc1 * (n-alpha+eps)) - sin(wc2 * (n-alpha+eps))  
      + sin(pi * (n-alpha+eps)) / (pi * (n-alpha+eps));  
Wr = boxcar(N);  
hn = hd * Wr;  
w = 0:0.01:pi;  
h = freqz(hn,1,w);  
plot(w/pi, abs(h));  
xlabel('normalised frequency');  
ylabel('magnitude');  
hold on;  
Wh = hamming(N);  
hn = hd * Wh;  
w = 0:0.01:pi;  
h = freqz(hn,1,w);  
plot(w/pi, abs(h), '-');  
Wh = hamming(N);  
hn = hd * Wh;  
w = 0:0.01:pi;  
h = freqz(hn,1,w);  
plot(w/pi, abs(h), '-');  
grid;  
hold off;
```

Result:

Thus the MATLAB for FIR filter design was performed and simulated and the output is obtained.